

Online Voting Sessions Application

Complete Project Documentation & Learning Guide

How the app is made, how it works, and the concepts behind every technology it uses

****Stack:**** MongoDB · Express · React · Node.js (MERN)

****Version:**** 2.0.0

****Author:**** KEERTHAN-089

****Date:**** October 2026

Table of Contents

Table of Contents	2
1. Introduction	5
2. Features at a Glance	5
3. Technology Stack	6
4. How a Voting Session Works	8
4.1 Session status	8
4.2 What keeps a vote fair.....	9
5. System Architecture	9
5.1 Development vs production	9
5.2 The life of a request.....	10
6. Project Structure.....	10
6.1 Key files and what they do	11
7. Backend Deep Dive	12
7.1 Startup and the middleware chain	12
7.2 Passwordless sign-in (the OTP flow).....	13
Requesting a code — POST /api/auth/request-code	13
Verifying a code — POST /api/auth/verify-code	14
7.3 Data models.....	15
Relationships	16
Key indexes	16
7.4 Session and vote endpoints (electionRoutes.js).....	17
Approvals and the voter cap — a race condition solved.....	19
7.5 The vote — one transaction, one ballot.....	20
The secret ballot	20
7.6 Results and CSV export.....	21
7.7 Images and selfies.....	21
7.8 Email	22
8. Frontend Deep Dive.....	23
8.1 Routes.....	23
8.2 Components	23
8.3 Global state and cross-cutting patterns	24
AuthContext — the only global store	24
useTheme — light / dark / system	25
usePolling — the app's "live" mechanism	25

api.js — the axios client	25
8.4 Design system	25
8.5 Accessibility	26
9. Database Design	26
Transactions and the replica-set requirement	27
Why documents instead of tables?	27
10. Complete API Reference.....	27
10.1 System	27
10.2 Auth (authRoutes.js).....	27
10.3 Sessions (electionRoutes.js)	28
10.4 Images (imageRoutes.js).....	29
11. Security Measures	29
11.1 Known limitations (honesty section)	30
12. Testing	31
12.1 What the tests verify	31
12.2 Load test, demo mode, and helpers.....	31
13. Running the Project Locally	32
13.1 Prerequisites.....	32
13.2 Setup steps	32
13.3 Sending sign-in codes by email.....	32
13.4 Environment variables.....	33
14. Deployment	33
14.1 Azure App Service.....	33
14.2 Render	34
14.3 Notes	34
15. Project History and Legacy Code	34
15.1 How the project evolved	34
15.2 Files that are NOT part of the live app	34
16. Concept Glossary	35
17. Ideas for Further Learning	37
Appendix A — npm Scripts.....	37
Root package.json	37
Backend package.json	38
Frontend package.json	38
Appendix B — Dependencies	38

Backend 38

Frontend 39

Closing Note..... 39

If the table of contents looks empty or out of date, click anywhere in it and press **F9** (or right-click → Update Field → Update entire table).

1. Introduction

This project is an **online voting application** built with the MERN stack (MongoDB, Express, React, Node.js). It lets anyone create a voting session — for a class leader election, a club president, a team captain, a poll — share it with voters through a QR code or link, collect one anonymous vote per person, and see the results live.

The app was built as a **learning project**. Its goal is not just to run a vote, but to exercise a full modern web-application stack: a REST API, a document database with transactions, token-based authentication, email delivery, browser camera access, and a component-based frontend.

This document is the complete written record of the project. It explains:

- **What the app does** — features, the lifecycle of a voting session, and the journeys of the organizer and the voter (sections 2 and 4).
- **How it is built** — the architecture, project structure, and every important file in the backend and frontend (sections 5–9).
- **Why it is built that way** — the concepts behind each technology, explained in plain language in the "Concept" boxes throughout.
- **How to run, test and deploy it** — local setup, automated tests, and cloud deployment (sections 12–14).

A quick-start guide with just the setup steps lives in [README.md](#); this document goes much deeper and doubles as a learning guide.

2. Features at a Glance

Feature	Description
Multiple simultaneous sessions	Many voting sessions can run at the same time. Each session is fully isolated: only its organizer can manage it, and only its approved voters can see its ballot.
Passwordless sign-in	No passwords at all. Users sign in with a 6-digit one-time code emailed to them. The first successful sign-in automatically creates the account.
QR code and join link	The organizer shares an 8-character join code, a join link, or a QR code. Scanning it on a phone opens the join page directly.
Voter limits	Each session has a maximum number of voters (1–5000). The limit is enforced by the database, so it can never be exceeded even under simultaneous joins.
Approval workflow	Organizers can pre-approve people through an email allow-list, or review and approve/reject joiners one by one from the Voters tab.
One vote per voter	A vote is recorded in a single database transaction that succeeds only for a voter who has not voted yet. Double-clicks, retries, or 50 simultaneous requests still count exactly once.
Secret ballot	The ballot is stored anonymously: it records the session and the chosen candidate, never the voter. The app cannot reveal who chose what.
Selfie options (per session)	No selfie (fastest), photo record (a selfie is taken with the vote), or face match (a face is enrolled when joining and re-checked before voting).

Feature	Description
Live or after-close results	Results can be visible to voters in real time, or hidden until the organizer closes the session.
CSV export	The organizer can download the final results as a CSV file for spreadsheets or record-keeping.
Responsive and accessible	Works on phones and desktops, with light/dark themes, keyboard navigation, and screen-reader support throughout.
Up to 50 candidates	Sessions support up to 50 candidates, each optionally with a photo, name, and party/tagline.

3. Technology Stack

The project is a **MERN** application: MongoDB, Express, React, and Node.js — one JavaScript language across the whole stack. The table below lists every technology used and what it does in this project.

Layer	Technology (version)	What it does here
Frontend UI	React 18 (<code>react ^18.2.0</code>)	Builds the interface out of reusable components. Bootstrapped with Create React App (<code>react-scripts 5.0.1</code>).
Frontend routing	react-router-dom v6	Maps URLs (<code>/dashboard</code> , <code>/join/ABC12345</code> , <code>/elections/:id/vote</code> , ...) to components without a page reload (a single-page app).
HTTP client	axios	Makes all API calls to the backend and automatically attaches the login token to every request.
Notifications	react-toastify	Small pop-up messages ("Vote submitted", "Link copied").
QR codes	qrcode.react	Renders the join QR code as an SVG on the Share tab.
Face recognition	@vladmandic/face-api 1.7	A computer-vision library that runs inside the browser : detects a face in the camera image and converts it to a 128-number "face descriptor" for face-match sessions.
Backend runtime	Node.js (18+)	Runs the server. JavaScript on the server means one language for the whole project.
Backend framework	Express 4	The web framework: defines routes, middleware, and request/response handling for the REST API.
Auth tokens	jsonwebtoken 9	Creates and verifies JWT login tokens.
Email	nodemailer 6	Sends the sign-in code by email over SMTP (works with Gmail).

Layer	Technology (version)	What it does here
File uploads	multer 2	Parses multipart form uploads (candidate photos, vote selfies) in memory.
Rate limiting	express-rate-limit 7	Limits how many requests an IP or a user can make per minute — protects sign-in and voting from abuse.
Security headers	helmet 7	Sets protective HTTP headers (content-security-policy is disabled here because the in-browser face model needs inline scripts and blob URLs).
Compression	compression 1.7	Gzips responses so the API is faster on slow connections.
Configuration	dotenv 16	Loads secrets and settings from a <code>.env</code> file into environment variables.
Database	MongoDB Atlas + Mongoose 7	The database (a document database) and the Mongoose ODM that defines schemas, models, and indexes.
Tests	node:test + supertest + mongodb-memory-server	Built-in Node test runner, HTTP testing, and a throwaway in-memory MongoDB (replica set) so tests never touch real data.
Dev tooling	concurrently, nodemon	Runs backend and frontend together in development; auto-restarts the backend on file changes.

Concept — What is the MERN stack?

MongoDB is a *document database*: instead of tables with rows, it stores JSON-like documents in collections. Data that belongs together (a session and its settings) lives in one document, which matches how JavaScript objects look. The project uses **Mongoose** to define schemas (the shape of each document), validation, and indexes.

Express is the most popular web framework for Node.js. It receives HTTP requests, runs them through middleware (authentication, rate limits, JSON parsing), and calls route handlers that read or write the database and send back JSON.

React is a frontend library that describes the interface as components — small, reusable pieces with their own state. React re-renders the page efficiently whenever data changes, instead of writing raw HTML for every screen.

Node.js is a JavaScript runtime outside the browser. It runs the Express server, connects to MongoDB, and sends email. Because backend and frontend are both JavaScript, types and conventions can be shared.

Concept — What is a REST API?

The frontend and backend communicate over HTTP using REST conventions: each resource (a session, a candidate, a vote) has a URL, and the HTTP method says what to do — **GET** reads, **POST** creates, **PUT** updates, **DELETE** removes. For example **POST /api/elections** creates a session, **GET**

`/api/elections/:id/results` reads its results. Both sides exchange JSON, the same data format JavaScript uses natively.

4. How a Voting Session Works

A voting session has a simple lifecycle: it is created as a **draft**, the organizer fills it with candidates and shares it, voting is **opened**, and finally it is **closed**. Here is the full journey step by step.

- 1. Create.** The organizer signs in (see section 7.2) and presses "Create a voting session". One form collects everything: what the vote is for, the maximum number of voters (default 60), whether voters take a selfie, and when results become visible. The organizer can optionally paste a list of email addresses that are allowed to vote. The session is created in **draft** status with a unique join code.
- 2. Add candidates.** On the Candidates tab the organizer adds at least two candidates (up to 50), each with a name, an optional party/tagline, and an optional photo. Photos are resized in the browser before upload. A session cannot be opened with fewer than two candidates.
- 3. Share.** The Share tab shows a QR code, the join link, and the 8-character join code. The organizer projects the QR code in class or sends the link to the group.
- 4. Voters join.** Each voter scans the code or opens the link, and signs in with their email (a one-time code is emailed to them). People on the organizer's email list are approved automatically; everyone else lands in the organizer's Voters tab as "waiting for approval". Approvals stop automatically at the voter limit.
- 5. Open voting.** When the organizer presses "Open voting" the session moves from **draft** to **open**. Each approved voter sees the ballot with all candidates, picks one, confirms, and (if configured) takes a selfie. Every voter gets exactly one vote.
- 6. Close.** The organizer closes the session, which is irreversible. Final results appear (with turnout), the organizer can download them as a CSV file, and — if selfies were used — can delete all of them with one button.

4.1 Session status

Status	Meaning	What can happen
draft	The session exists but voting has not started.	Organizer: edit settings, add/remove candidates, open voting. Voters: can join and wait, but see no ballot.
open	Voting is in progress.	Approved voters cast their one vote. Organizer: can fix candidate names/photos but not add or remove candidates, and cannot change the selfie mode (voters have joined).
closed	Voting is over — permanently.	No more votes. Results are final (and become visible if they were hidden). Organizer: export CSV, delete selfies, or delete the whole session.

4.2 What keeps a vote fair

- **One vote each.** A vote is recorded in a single database transaction that only succeeds for a voter who has not voted yet — the database itself, not the code, is the final gate. Double-clicking or sending many requests at once still counts once (section 7.5 explains the mechanism).
- **Voter limit.** Approvals stop at the configured maximum, so there can never be more voters — or votes — than allowed, even if many people join at the same instant (section 7.4).
- **Secret ballot.** The app stores *that* a voter has voted, and separately stores an *anonymous* ballot. It never stores who a voter chose. (Someone with direct access to the database server could still study its logs for timing; the app itself cannot reveal a choice.)
- **Separate sessions.** Only the creator of a session can manage it, and only voters approved for that session can see its ballot or vote in it. Sessions are invisible to each other.
- **Selfie accountability (optional).** Photo-record sessions attach a selfie to each vote, and face-match sessions verify the face matches the one enrolled at join time. Selfies live on the voter's record — never on the ballot — and only the organizer can view them.

5. System Architecture

The app is a **single Node.js service**. In production, the backend does two jobs: it serves the REST API under `/api`, and it also serves the built frontend (the compiled React app) to browsers. This means one server, one deployment, and no cross-origin issues.



5.1 Development vs production

- **Development.** Two servers run together (`npm run dev`): the backend (Express) on port **3000** and the Create React App dev server on port **3001**. The frontend's `package.json` has `"proxy": "http://localhost:3000"`, so any request to `/api` from the React dev server is forwarded to the backend automatically. That is why the browser only ever talks to port 3001 and there is no CORS configuration anywhere.
- **Production.** `npm run build` compiles the React app into static files in `frontend/build`. The backend serves those files directly (plus a fallback that returns `index.html` for any non-API path, so deep links like `/join/ABC12345` work after a reload). One Node process is the whole product — which is also what the deployments in section 14 rely on.

5.2 The life of a request

The browser loads the app: in production from Express serving `frontend/build`; in development from the Create React App dev server on port 3001.

The user interacts with the React app. Every data operation is an axios call to `/api/...`

The request reaches Express and passes through middleware in order: proxy/IP handling → security headers (helmet) → compression → JSON body parsing → the matching route (see section 7.1 for the exact chain).

The route handler validates the request, checks the JWT token (attached automatically as `Authorization: Bearer <token>`), and talks to MongoDB through Mongoose.

The response comes back as JSON. The React component updates its state, and the UI re-renders. Live updates (pending approvals, new results) are achieved by **polling** — refetching every few seconds — described in section 8.3.

Concept — What is a proxy? What is an SPA fallback?

A **proxy** is an intermediary that forwards requests. In development, the React dev server acts as a proxy: when the browser asks it for `/api/elections`, it silently forwards the request to the backend on port 3000 and passes the answer back. This makes the two servers look like one origin to the browser.

An **SPA fallback** solves a routing problem: the React app is one HTML page, and all its "pages" (like `/join/ABC12345`) exist only in JavaScript. If someone opens that URL directly, the server has no such file — so Express answers every non-API path with `index.html`, and React then renders the right page from the URL. The catch-all route `app.get('*')` in `backend/app.js` does exactly this.

6. Project Structure

The repository has two main folders — `backend/` (the Express API) and `frontend/` (the React app) — plus deployment tooling at the root. The tree below shows the files of the **live** app.

Voting_app/	
└ backend/	Express API server (Node.js)
└ server.js	entry point: env, DB connect, start
└ app.js	the Express app: middleware + routes
└ jwt.js	JWT helpers + auth middleware
└ access.js	election loading + organizer/voter checks
└ clientAddress.js	normalizes client IPs for rate limits
└ mailer.js	sends the sign-in code email
└ models/	
└ user.js	user = email + display name
└ election.js	a voting session
└ candidate.js	a candidate in a session
└ membership.js	voter <-> session link + approval + hasVoted
└ vote.js	the anonymous ballot
└ otpCode.js	the sign-in code (hashed, expiring)
└ image.js	candidate photo or selfie (stored in Mongo)
└ routes/	
└ authRoutes.js	sign-in endpoints
└ electionRoutes.js	ALL session/vote endpoints (largest file, 821 lines)
└ imageRoutes.js	serves candidate photos & selfies
└ scripts/	demo.js, load-test.js, reset-db.js, test-email.js
└ test/api.test.js	~28 automated API tests
└ .env / .env.example	secrets & settings (never committed)
└ frontend/	React single-page app
└ public/	index.html, icons, /models (face model files)
└ scripts/copy-face-models.js	copies face-api models into public/
└ src/	
└ index.js	React entry point
└ App.js	all routes + global layout
└ components/	16 components (section 8)
└ styles/	design tokens + page stylesheets
└ utils/	api client, auth context, theme, polling
└ build/	production build (served by the backend)
└ deploy/azure.js	Azure App Service deployment automation
└ render.yaml	Render.com deployment definition
└ package.json	root scripts: dev, demo, test, azure:deploy...
└ README.md	quick-start guide

6.1 Key files and what they do

File	What it does
backend/server.js	Startup: loads <code>.env</code> , connects to MongoDB (exits on failure so the host restarts it), checks the JWT secret, warns about missing email setup or legacy data, starts listening, handles graceful shutdown.
backend/app.js	Builds the Express app: middleware chain, mounts the three route groups, <code>/api/health</code> , the error handler, and — in production — serves <code>frontend/build</code> .
backend/jwt.js	Signs and verifies JWTs (12-hour expiry) and exports the auth middleware (<code>jwtAuthMiddleware</code> , <code>optionalAuth</code>).
backend/access.js	Authorization helpers: loads the session by <code>:id</code> into <code>req.election</code> , and gates routes with <code>requireOrganizer</code> / <code>requireApprovedVoter</code> .
backend/clientAddress.js	Strips the connection port from <code>req.ip</code> (some hosts report <code>1.2.3.4:56789</code>), so rate limiting counts one visitor as one visitor.

File	What it does
<code>backend/mailer.js</code>	The only email in the app: the sign-in code. Uses nodemailer/SMTP; prints the code to the server console when email is not configured.
<code>backend/routes/authRoutes.js</code>	Sign-in endpoints: request a code, verify a code, read/update my profile.
<code>backend/routes/electionRoutes.js</code>	Everything about sessions: create, lists, join, ballot, vote, results, and the full organizer management suite. The heart of the app.
<code>backend/routes/imageRoutes.js</code>	Serves stored images: candidate photos publicly (cached), selfies only to the session's organizer.
<code>frontend/src/App.js</code>	The route table: which URL shows which component, plus global layout, theme and navigation.
<code>frontend/src/Utils/api.js</code>	The axios instance all components use: attaches the token, handles expired logins, formats error messages.
<code>frontend/src/Utils/AuthContext.js</code>	Global auth state: keeps the signed-in user, restores the session from localStorage on load, exposes sign-in/sign-out.
<code>frontend/src/styles/tokens.css</code>	The design system: colors, typography, spacing, shadows, and dark-mode tokens as CSS variables.
<code>backend/test/api.test.js</code>	The automated test suite (section 12).
<code>deploy/azure.js</code>	Scripts <code>azure:setup</code> , <code>azure:deploy</code> , <code>azure:package</code> that automate Azure App Service deployment (section 14).

A note on leftover files. The repository also contains files from the first version of this app (Aadhaar-number and password login). They are **not** loaded by the live server — `app.js` imports only the files above. Section 15 lists them in detail.

7. Backend Deep Dive

The backend is a classic Express application: a startup file, an app definition, models, and route modules. This section walks through each piece, with the concepts explained along the way.

7.1 Startup and the middleware chain

`server.js` starts everything, in this order:

Load environment variables with `dotenv` (from `backend/.env`).

Build the Express app by requiring `app.js` (all middleware and routes register there).

Fail immediately if `JWT_SECRET` is missing — tokens cannot be signed safely without it.

Connect to MongoDB (30 s timeout, connection pool of 50). If the connection fails, the process **exits** so the hosting platform restarts it — a crashed-but-restarting server is better than a running server with no database.

Warn if SMTP is unconfigured (sign-in codes will print to the terminal instead) and if legacy v1 collections are detected in the database.

Start listening on `PORT` (default 3000). On `SIGTERM`/`SIGINT` it shuts down gracefully: stops accepting connections, disconnects from MongoDB, exits cleanly (forced exit after 10 s).

`app.js` then processes **every request** through this middleware chain, in order:

#	Middleware	Why
1	<code>app.set('trust proxy', 1)</code> (production)	In production a proxy (Render/Azure) sits in front; trusting it by one hop makes <code>req.ip</code> report the real client IP instead of the proxy's.
2	<code>normalizeClientAddress</code>	Strips the connection port from <code>req.ip</code> so rate limits see one visitor as one visitor.
3	<code>helmet</code> (CSP disabled)	Sets security headers. Content-Security-Policy is turned off deliberately: the React build and the in-browser face model need inline scripts and blob URLs.
4	<code>compression</code>	Gzips API responses.
5	<code>express.json({ limit: '400kb' })</code>	Parses JSON bodies; 400 kb is large enough for a compressed selfie data-URL but small enough to discourage abuse.
6	Route mounts	<code>/api/auth</code> , <code>/api/elections</code> , <code>/api/images</code> .
7	<code>GET /api/health</code>	200 <code>{status:"ok", database:"connected"}</code> when MongoDB is ready, 503 otherwise — used by hosting health checks and the Azure deploy script.
8	<code>/api</code> 404 catch-all	Any unknown <code>/api</code> path gets a JSON 404.
9	Static + SPA fallback	If <code>frontend/build</code> exists, serves it and answers every other path with <code>index.html</code> .
10	Error handler	Translates errors into clean JSON: file too large → 400 "Image is too large...", body too large → 413, bad JSON → 400, Mongoose validation → 400 with an <code>errors</code> array, anything else → 500 (logged).

Concept — What is middleware?

In Express, middleware is a function that receives (`req`, `res`, `next`) and runs on every request (or on matching routes). Each one does one job — parse JSON, check the token, limit the rate — then either sends a response or calls `next()` to pass the request along. The order in which they are registered is the order in which they run, which is why the chain above is so explicit.

7.2 Passwordless sign-in (the OTP flow)

The app has **no passwords**. A user "signs in" by proving they control their email inbox: the server emails a 6-digit code, and entering it correctly returns a login token. First successful sign-in automatically creates the account — there is no separate registration.

This is why the `User` model is tiny: just an email and a display name. Identity = email, proof = one-time code.

Requesting a code — POST `/api/auth/request-code`

Validate the email (format, ≤ 254 characters).

Rate-limit: one code per email per minute (a second request within 60 s gets a 429 with `retryAfterSeconds`).

Generate a 6-digit code with `crypto.randomInt(0, 1000000)` (cryptographically secure).

Hash the code before storing: `HMAC-SHA256(key = JWT_SECRET, message = "email:code")`. Only the hash is saved, so a database leak never reveals live codes.

Upsert an `OtpCode` document: hash, `attempts: 0`, `sentAt`, `expiresAt = now + 10 minutes`. A TTL index deletes it automatically when it expires.

If the sign-in came from a join link, look up the session (using its join code) so the email can say "Sign in to vote in *Class Leader Election*".

Send the email. **If sending fails, delete the OTP** and return 502 — this lets the user retry immediately instead of waiting out a code that was never delivered.

Verifying a code — POST /api/auth/verify-code

Validate email + 6-digit code format.

Atomically count the attempt: one database update does "increment `attempts`, but only if the code is unexpired and has fewer than 5 attempts". Because it is a single conditional update, even parallel guesses cannot exceed the limit. No matching document → 429 if a valid code exists (locked out) or 400 (expired).

Compare the hash with `crypto.timingSafeEqual` — a constant-time comparison that does not leak how many characters matched.

Wrong code → 401 with `attemptsLeft` (the frontend shows "Wrong code. N attempts left."). After 5 wrong tries the code is dead.

Correct code → delete the OTP (each code is **single-use**), upsert the user by email (creating it on first sign-in), and return `{ user, token }`.

The token is a **JWT** signed with `JWT_SECRET`, valid for **12 hours**, containing `{ id, email }`. The frontend stores it in `localStorage` and sends it as `Authorization: Bearer <token>`; `jwtAuthMiddleware` verifies it and attaches `req.user` to the request.

Two rate-limit layers protect this flow: 600 requests / 15 min **per IP** (loose, because a classroom shares one Wi-Fi IP), plus the tighter per-email limits above (1 code/min, 5 tries per code).

Concept — What is an OTP (one-time password)?

An OTP is a short code that works once and expires quickly. The user proves they own the email account, and the code becomes useless afterwards — so even if it leaks (a shoulder-surfer sees it), it expires in 10 minutes and only ever grants one login. Compared to passwords, OTPs cannot be reused across sites, are never "remembered" by the user, and there is nothing for the app to store that an attacker could use to log in later.

Concept — What is HMAC, and why hash the code?

HMAC (Hash-based Message Authentication Code) computes `HMAC(key, message)` — a fingerprint of a message that is unforgeable without the key. The app stores `HMAC-SHA256(JWT_SECRET, "email:code")` instead of the raw code. If the database is stolen, the attacker finds only hashes; guessing the 6-digit code for one specific email still requires 5 attempts before the account is locked out. Note that hashing the code is possible **because** the code space is tiny and the app controls the

comparison — passwords normally need slow salted hashes (bcrypt), but a 6-digit code with attempt limits works differently.

Concept — Why a timing-safe comparison?

A normal string comparison stops at the first different character, so the time it takes reveals how many characters were correct — an attacker could guess a code character by character.

`crypto.timingSafeEqual` compares in constant time, so every wrong guess costs the same and leaks nothing.

Concept — What is a JWT (JSON Web Token)?

A JWT is a signed token in three parts: `header.payload.signature`. The payload here is `{ id, email, exp }` — small, readable, and **self-contained**: the server can verify who the user is from the token alone, without a session store. The signature proves the server (the only holder of `JWT_SECRET`) issued it and that nobody tampered with it. The trade-off: once issued, a JWT stays valid until it expires (12 h here), because the server does not remember it. When the token expires or is removed, the frontend automatically sends the user back to sign-in.

Concept — What is rate limiting?

Rate limiting caps how many requests a client may make in a time window — here, per IP for the sign-in endpoints and per signed-in user for everything else (300/min generally, 20/min for voting, 20/hour for creating sessions). It makes brute-force guessing and accidental double-submits (a user spamming refresh) slow to impossible. Note the design choice: per-user limits on the election routes instead of per-IP, because many users can share one classroom IP.

7.3 Data models

Seven Mongoose models define everything the app stores. A "model" wraps a MongoDB collection with a schema — field names, types, validation, and indexes.

Model (collection)	Fields	Purpose
User (users)	<code>email</code> (unique, lowercase), <code>name</code> (≤ 80 chars, default ""), timestamps	An account. That is all — sign-in is by emailed code, so there is no password, role, or Aadhaar field.
Election (elections)	<code>title</code> , <code>description</code> , <code>organizer</code> ($\rightarrow$ User), <code>status</code> (draft/open/closed), <code>joinCode</code> (unique), <code>maxVoters</code> (1–5000), <code>approvedCount</code> , <code>selfieMode</code> (none/photo/faceMatch), <code>resultsVisible</code> (live/afterClose), <code>allowList</code> (hidden), <code>closedAt</code>	One voting session. <code>approvedCount</code> is a running counter that makes the voter cap enforceable atomically.
Candidate (candidates)	<code>election</code> ($\rightarrow$ Election), <code>name</code> , <code>party</code> (shown as "Group / tagline"), <code>image</code> ($\rightarrow$ Image, optional)	A candidate in one session. Deliberately stores no vote totals — totals are always counted from the Vote collection, so concurrent votes never contend on one document.

Model (collection)	Fields	Purpose
Vote (votes)	election (→ Election), candidate (→ Candidate). Random _id , no timestamps , no voter field	The anonymous ballot. It says which session and which candidate — never who. The random (not time-based) id and missing timestamp stop timing analysis from correlating a vote with a voter.
Membership (memberships)	election , user , status (pending/approved/rejected), approvedVia (list/organizer), hasVoted , votedAt , enrolSelfie/voteSelfie (→ Image), faceDescriptor (hidden)	The link between a voter and a session: approval state, whether they voted, and their selfies. Stores <i>*that*</i> someone voted, never <i>*what*</i> they chose.
OtpCode (otpcodes)	email (unique), codeHash , attempts , sentAt , expiresAt (TTL index)	The current sign-in code for one email, as a hash. MongoDB deletes it automatically when expiresAt passes.
Image (images)	election , kind (candidate/selfie), contentType , data (Buffer)	Candidate photos and selfies, stored in MongoDB (not on disk) so they survive deploys on hosts without a persistent disk. Sizes are capped (1 MB candidate, 200 KB selfie).

Relationships

User 1 —organizes— n Election
 Election 1 —has— n Candidate (each may have 1 Image, kind=candidate)
 Election 1 —has— n Membership (each links 1 User; optional selfie Images)
 Election 1 —has— n Vote (each points at 1 Candidate; never at a User)
 OtpCode — one per email, exists only while the code is valid

Key indexes

Index	On collection	What it enforces / enables
unique email	users	One account per email.
unique joinCode	elections	Each join code resolves to exactly one session.
unique { election , user }	memberships	A user can join a session only once — the backbone of "one vote per voter".
{ election , status }, user	memberships	Fast lookups for the approval queue and "my sessions".
TTL expiresAt	otpcodes	MongoDB auto-deletes expired codes — no cleanup job needed.
{ election , candidate }	votes	Fast vote counting per session.
organizer , election	elections / candidates / images	Fast "my sessions" lists and candidate/photo lookups.

Concept — What are schema, model, index — and select:false?

A **schema** describes the shape of a document (fields and rules); a **model** is the object you use to create/read/update those documents (e.g. `Election.findById(...)`). An **index** is a database-side lookup table for a field (like a book index): without one, finding "all votes for session X" scans every document; with one it jumps straight to the matches. `select: false` on a field means Mongoose will not include it in query results unless the code explicitly asks (`.select('+field')`) — the app uses this for the email allow-list and the face descriptors, so those sensitive fields never leak through an ordinary read.

Concept — Why a replica set? What is a transaction?

MongoDB **transactions** (grouping several writes into one all-or-nothing operation) are only available on a **replica set** — a cluster of 3+ database nodes that keep copies of the data and elect a leader. MongoDB Atlas provides this out of the box, which is why the app requires Atlas (or the in-memory replica set used by the demo/tests) rather than a plain local **mongod**. A transaction is **atomic**: either every write in it happens, or none do — no in-between state is ever visible. Section 7.5 shows where that matters.

7.4 Session and vote endpoints (electionRoutes.js)

`electionRoutes.js` is the largest file in the project (821 lines) and contains every endpoint for sessions, voters, candidates, votes, and results. Three rate limiters apply here, keyed per signed-in user: 300 requests/min general, 20/min on voting, 20/hour on creating sessions.

One public endpoint exists before the auth wall — the join page must work for signed-out visitors:

Method + path	Access	Purpose
GET <code>/api/elections/join/:code</code>	Anyone (optional token)	The join page data: public session info, whether it is full, whether I am the organizer, and my membership status.

Everything below requires a valid JWT (401 otherwise). The organizer-only routes additionally run `requireOrganizer`, and the voting routes run `requireApprovedVoter` — both defined in `access.js`.

Method + path	Access	Purpose
POST <code>/api/elections</code>	Signed in	Create a session (creator becomes the organizer). Validates all fields, generates a unique 8-character join code (alphabet without confusable I/O/0/1; retries up to 5 times on collision), and caps each organizer at 10 active sessions.
GET <code>/api/elections/organizing</code>	Signed in	Sessions I run, with pending/voted counts (computed with a small aggregation).
GET <code>/api/elections/voting</code>	Signed in	Sessions I joined as a voter, with my membership status and whether I voted.

Method + path	Access	Purpose
POST /api/elections/join/:code	Signed in	Join a session. Idempotent (joining twice returns the same membership). Face-match sessions require a 128-number face descriptor + selfie at join. Emails on the allow-list are auto-approved.
GET /api/elections/:id/ballot	Approved voter	The ballot: session info + candidates without vote counts .
POST /api/elections/:id/vote	Approved voter	Cast the vote (full flow in section 7.5).
GET /api/elections/:id/results	Organizer / approved voter (when visible)	Vote counts (section 7.6).
GET /api/elections/:id/manage	Organizer	Everything the organizer dashboard needs: session, allow-list, candidates, pending/voted counts.
PUT /api/elections/:id/manage	Organizer	Edit settings. selfieMode locks once anyone has joined; maxVoters can never drop below the current approved count.
POST /api/elections/:id/manage/open	Organizer	draft → open. Requires at least 2 candidates.
POST /api/elections/:id/manage/close	Organizer	open → closed (irreversible). Sets closedAt and clears the results cache.
DELETE /api/elections/:id/manage	Organizer	Delete the session and everything in it (votes, memberships, candidates, images) in a cascading delete.
POST /api/elections/:id/manage/candidates	Organizer	Add a candidate — draft only . Accepts an optional photo (multipart, ≤ 1 MB, jpeg/png/webp). Max 50 candidates.
PUT /api/elections/:id/manage/candidates/:candidateId	Organizer	Edit name/party/replace photo (old photo deleted). Allowed even while open — but never after close. Clears the results cache.
DELETE /api/elections/:id/manage/candidates/:candidateId	Organizer	Remove a candidate — draft only. Deletes its photo too.

Method + path	Access	Purpose
GET /api/elections/:id/manage/voters	Organizer	The voter roster with selfie ids — the face descriptors are never sent to any client.
POST /api/elections/:id/manage/voters/approve-all	Organizer	Approve pending voters oldest-first until the cap. Returns how many were approved and whether the session is now full.
POST /api/elections/:id/manage/voters/:membershipId/approve	Organizer	Approve one voter; 409 when the session is full.
POST /api/elections/:id/manage/voters/:membershipId/reject	Organizer	Reject a voter (frees their slot — but not after they have voted).
POST /api/elections/:id/manage/allow-list	Organizer	Add emails to the allow-list (parses commas/spaces/newlines, validates, dedupes) and auto-approves anyone already waiting who is on the new list.
POST /api/elections/:id/manage/purge-selfies	Organizer	After close only: permanently deletes every selfie and face descriptor in the session (privacy cleanup).
GET /api/elections/:id/manage/results.csv	Organizer	Download final results as CSV (section 7.6).

Approvals and the voter cap — a race condition solved

The voter cap must hold **even when 20 people join at the same instant**. A naive check ("read the count, if below max, add one") fails under concurrency: two requests can both read "4 of 5" and both approve, ending at 6. The app instead claims a slot with **one conditional atomic update**:

```
Election.findOneAndUpdate(
  { _id, status: { $ne: 'closed' },
    $expr: { $lt: ['$approvedCount', '$maxVoters'] } }, // still room?
  { $inc: { approvedCount: 1 } }                       // then take the slot
)
```

MongoDB evaluates the condition and applies the increment as a single operation — only one of any number of simultaneous requests can match. If the membership turned out to be already handled, the slot is handed back (**\$inc -1**). The test suite proves it: 20 simultaneous approvals against a limit of 5 produce exactly 5 approved voters.

The same pattern appears everywhere a state change must be race-proof: open/close use **findOneAndUpdate** filtered on the expected current status, and — most importantly — voting uses it (next section).

Concept — What is a race condition?

A race condition is a bug that depends on timing: two requests read the same state, both act on it, and the second overwrites the first — e.g. two approvals of the last slot, or two votes from one voter. The fix used throughout this app is to make the read-and-act a **single conditional database update**: the condition and the change are evaluated together, atomically, so there is no gap for a second request to slip into.

7.5 The vote — one transaction, one ballot

`POST /api/elections/:id/vote` is the most carefully engineered endpoint in the app. Server-side, step by step:

Rate-limit (20 votes/min/user), then `requireApprovedVoter`: the caller must have an approved membership in this session.

The session must be `open` — otherwise 409 with "Voting has not started yet" / "Voting is closed".

If `membership.hasVoted` is already true → 409 "You have already voted".

The `candidateId` must be a valid id **of a candidate in this session** (not just any candidate anywhere).

Face-match sessions: reload the membership with the hidden `faceDescriptor`, compute the Euclidean distance between it and the descriptor the browser just computed from the live photo, and refuse (403) if the distance is above the threshold — a different face.

Photo/face-match sessions: save the vote selfie (a data-URL under 200 KB) as an Image document.

The atomic transaction — both of these commit together, or neither does:

```
// 1. Claim the vote: matches only if not voted yet.
// Under concurrency, exactly ONE request matches.
Membership.findOneAndUpdate(
  { _id, status: 'approved', hasVoted: false },
  { hasVoted: true, votedAt, voteSelfie },
  { session } // inside the transaction
)

// 2. The anonymous ballot – same transaction.
Vote.create([ { election, candidate } ], { session })
```

If anything fails, the transaction rolls back and the orphaned selfie is deleted. If the update matched nothing (someone else's request won the race), the answer is 409 — the frontend shows "You have already voted".

Why this is correct: the *condition* on the update (`hasVoted: false`) is evaluated atomically with the change, so of 50 simultaneous vote requests exactly one matches and exactly one `Vote` document is created — the test suite verifies precisely that (50 parallel requests → one 200, one ballot row). The ballot and the "has voted" flag live in the same transaction, so it is impossible to have a ballot without a marked voter, or a marked voter without a ballot.

The secret ballot

- A `Vote` document has **no voter id**. It names the session and the candidate, nothing else.
- Its `_id` is generated from `crypto.randomBytes(12)`, not from the time-ordered ObjectId generator, and the model sets `versionKey: false` and **no timestamps** — so votes carry no order information an analyst could line up against "who voted when".

- The voter's identity lives only on the Membership: `hasVoted` and `votedAt`. Selfies and the face descriptor are also stored on the Membership — the ballot stays clean.
- Honest caveat (also in the README): someone with direct access to the database server could still study its logs for timing correlations. The **application itself** cannot reveal a choice — there is no query that joins votes to voters.

Concept — What does "atomic" mean? (ACID transactions)

Atomicity is the "A" in ACID: a group of operations either all succeed or all fail — observers can never see a half-finished state. Here, "mark voter as having voted" + "store ballot" are one transaction: if the ballot write failed, the voter would not be marked; if the voter claim failed, no ballot exists. Without this, a crash between the two writes could either lose a ballot or let the voter vote twice on retry. Transactions also require the replica set mentioned in section 7.3.

7.6 Results and CSV export

Vote totals are **never stored** — they are always computed fresh from the `votes` collection with an aggregation pipeline: `$match` votes of this session → `$group` by candidate summing 1 → merge with the candidate list (zero-filling candidates nobody chose) → sort descending. The response is `{ candidates, totalVotes }`, and the frontend adds turnout from `approvedCount`.

Because every open results page polls every 5 s, counting is cached: an in-memory `resultsCache` Map stores the **promise** of each computation for 3 seconds, so all pollers hitting at the same moment share one count. The cache clears itself on close/edit/delete and is capped at 500 entries; a rejected promise evicts itself.

Visibility rules for `GET /:id/results`: the organizer can always see results. Everyone else must be an approved voter, and if the session was configured with `resultsVisible: "afterClose"`, non-organizers get 403 `{hidden: true}` until the organizer closes the session — the frontend shows "Results will appear when the organizer closes voting."

The CSV export writes one row per candidate. Every cell passes through `csvCell`, which prefixes `'` when a value starts with `=`, `+`, `-` or `@` — a classic defense: a candidate literally named `=SUM(A1:A99)` must not become a live spreadsheet formula when the organizer opens the file in Excel.

Concept — What is an aggregation pipeline? What is CSV formula injection?

An **aggregation pipeline** is a sequence of stages (`match` → `group` → `sort`) that MongoDB runs over documents to compute results — here, a grouped vote count. It is the database equivalent of a `GROUP BY` query and is far more efficient than loading every vote into Node and counting there.

CSV formula injection exploits that spreadsheets execute cells starting with `=`, `+`, `-`, `@` as formulas. If an untrusted value (like a candidate name) ends up in a CSV cell unescaped, opening the file could run an attacker-chosen formula. Escaping those characters keeps exported data inert.

7.7 Images and selfies

- Images are stored **in MongoDB** as binary Buffers with their content type — not on disk and not in GridFS. On hosts like Azure's free tier, the disk is wiped on redeploy; the database is not. The sizes are small enough (≤ 1 MB per candidate photo, ≤ 200 KB per selfie) that this stays cheap.

- `GET /api/images/:id` decides access by `kind`: **candidate** photos are public and sent with `Cache-Control: public, max-age=604800, immutable` (a replaced photo gets a new id, so the old one may be cached forever). **Selfies** require sign-in, require being the organizer of that image's session, and are sent with `Cache-Control: private, no-store`.
- Uploads go through multer in memory with type checks (jpeg/png/webp only) and size limits; the frontend additionally downscales images in the browser before uploading (section 8.7).

Selfie mode	What happens
<code>none</code>	Sign-in and approval only. Fastest; no camera involved.
<code>photo</code>	Each voter takes a selfie at vote time. The organizer can view it if a vote is disputed.
<code>faceMatch</code>	Each voter takes a selfie when joining (enrolment). At vote time the browser computes a face descriptor from a fresh photo, and the server compares it to the enrolled one — a different face is refused.

How face matching works. The `face-api` library runs in the browser: it detects the face, finds 68 landmarks, and produces a **128-number descriptor** that encodes the face's geometry. The browser sends only those numbers (plus the photo) — the server never compares photos, only descriptor distances (Euclidean distance; default threshold `FACE_MATCH_THRESHOLD = 0.5`, lower = stricter). The descriptor is stored with `select: false` and is never serialized in any API response.

Honest limitation. Face match is a deterrent, not proof of identity: it compares a live photo to the joining photo and has **no liveness check**, so a good printed photo of the voter could pass. The README states this openly.

Privacy. After closing, the organizer can purge every selfie, descriptor, and reference with one button (`purge-selfies`).

7.8 Email

- Sent with **nodemailer over SMTP** — no third-party email API. Configured for Gmail: `SMTP_PORT` 587 (or 465 → TLS), `SMTP_USER` + `SMTP_PASS` (a 16-letter Google **app password**, whitespace-stripped — Google displays them in 4-character groups), optional `MAIL_FROM`.
- Exactly **one email** exists in the app: the sign-in code, sent from "Voting App <SMTP_USER>". If the sign-in came from a join link, the email names the session and its organizer.
- When SMTP is not configured (`smtpConfigured()` false), the server prints the code to its console instead — the local-development fallback.
- **Anti-phishing hardening:** session titles and organizer names (user-typed text that will be emailed to arbitrary addresses) are stripped of control characters/newlines, truncated to 80 characters, and any text matching a link pattern (`https?:`, `www.`, domain-like strings) is discarded — a link smuggled into a session title falls back to the plain email instead of being sent to strangers. These exact behaviors are covered by tests.

Concept — What is SMTP? What is an app password?

SMTP (Simple Mail Transfer Protocol) is the standard protocol for sending email. Sending directly from a server is unreliable (spam filters), so the app delegates to Gmail's SMTP server, which delivers the message with Gmail's reputation.

An **app password** is a generated 16-character password that grants one app access to a Google account without the real password — and without needing to disable 2-Step Verification (which must be **on** for app passwords to exist). Using a dedicated mailbox for the app keeps voters' replies and sending limits separate from a personal inbox.

8. Frontend Deep Dive

The frontend is a React single-page app bootstrapped with Create React App. There is no Redux, no TypeScript, and no CSS framework — plain CSS with a design-token system, one global auth context, and local component state.

8.1 Routes

Path	Component	Guard	Purpose
/	Home	—	Landing page: hero, "Create a session" card, and a "Join with a code" form that navigates to <code>/join/:code</code> .
/login	Login	—	Email OTP sign-in. Honors <code>?next=</code> (sanitized to site-relative) so users return to where they were headed.
/join/:code	JoinElection	— (self-gates)	The page a QR code or link opens: session info, sign-in prompt, join request, selfie enrolment, and automatic onward navigation.
/dashboard	Dashboard	RequireAuth	The signed-in home: "Sessions I organize" and "Sessions I vote in" cards with live counts.
/create	ElectionForm	RequireAuth	Create a new session.
/elections/:id/manage	ElectionManage	RequireAuth	The organizer console with Share / Candidates / Voters / Results tabs.
/elections/:id/vote	VotingPage	RequireAuth	The ballot.
/elections/:id/results	Results	RequireAuth	Live or final results.
*	→ Navigate to /	—	Catch-all redirect.

`RequireAuth` redirects signed-out visitors to `/login?next=<where they were going>`, and wraps signed-in users in `NameGate` — a one-time "What is your name?" form that saves the display name (used in the approval queue) via `PUT /auth/me`.

8.2 Components

Component	What it does
Home	Static landing page: features, "how it works" steps, session-code input that jumps to <code>/join/:code</code> .

Component	What it does
Login	Two-step OTP form (email → code) with a 60-second resend countdown, attempts-left messaging, and the <code>joinCode</code> pass-through so the email can name the session.
Navbar	Brand, links, theme toggle, sign-out; hamburger menu on small screens (closes on route change, Escape, or outside click).
Dashboard	Both session lists side by side, polled every 10 s. Per-session status badges and the right CTA for each state (vote now / waiting for approval / view results / manage).
JoinElection	The QR/join-link destination. Auto-joins when possible (signed in, has a name, session open, not face-match — face enrolment needs an explicit selfie), polls every 4 s while waiting for approval, and jumps to the ballot the moment the session opens.
VotingPage	The ballot: radio-group candidate cards (photo, name, party), a choose → confirm → selfie flow, one-vote warnings, and success animation. Polls every 5 s while the session is still <code>draft</code> so it flips to "voting open" by itself.
Results	Voter-facing results page: polls every 5 s until the session closes, shows the "results hidden until close" notice when configured, then renders the board.
ResultsBoard	Pure presentational ranking: turnout, leader/winner (tie-aware), ranked candidates with percentages and colored bars. Shared by <code>Results</code> and the organizer's Results tab.
ElectionForm	Create/edit session form: title, description, max voters (1–5000), selfie mode, results visibility, allow-list. Used standalone on <code>/create</code> and inline in the manage Share tab (where the selfie lock is respected).
ElectionManage	The organizer console: tab state in the URL (<code>?tab=</code>), 10 s polling, open/close/delete actions with confirm dialogs, QR code + copy-link, and the CSV download.
VotersTab	Approval queue and roster: search, approve/reject per row, approve-all, allow-list bulk add, capacity meter ("N of max places taken"), selfie thumbnails, and the post-close purge-selfies button.
CandidateForm	Add/edit candidate with image preview. Images are downscaled in-browser (<code>resizeImageFile</code> , ≤ 600 px, JPEG 0.85) before the multipart upload.
SelfieCapture	Camera capture via <code>getUserMedia</code> . For face-match sessions it lazy-loads the face-api library and models from <code>/models</code> and computes the 128-number descriptor from the captured frame.
AuthImage	Displays a selfie, which needs the Bearer token (a plain <code></code> cannot send headers): fetches the blob through axios, turns it into an object URL, links to the full-size view.
RequireAuth	The auth gate + <code>NameGate</code> described above.
Icon	~30 inline SVG icons (stroke-based, inherit <code>currentColor</code>), replacing the Font Awesome CDN the first version used.

8.3 Global state and cross-cutting patterns

AuthContext — the only global store

`AuthProvider` wraps the whole app and exposes `{ user, loading, signIn, signOut, setUser }`. The token lives in `localStorage`; on first load the app calls `GET /auth/me` to restore the user. The axios client

(`utils/api.js`) attaches `Authorization: Bearer <token>` to every request and, on any 401 while a token exists, clears it and fires a window event (`auth:expired`) that the context listens for — one place, and the whole app flips to signed-out. `RequireAuth` then bounces the user to `/login?next=...` so no work is lost.

useTheme — light / dark / system

A three-state theme (light, dark, system) stored in `localStorage`. "System" removes the `data-theme` attribute so CSS `prefers-color-scheme` rules apply; an explicit choice stamps `data-theme` on `<html>`. A tiny inline script in `index.html` applies the saved theme **before first paint**, preventing a flash of the wrong theme.

usePolling — the app's "live" mechanism

`usePolling(fn, ms, enabled)` runs `fn` every `ms` with `setInterval`, **skips ticks while the browser tab is hidden**, and keeps the latest callback in a ref so closures never go stale. This one hook powers every "live" behavior: dashboard counts (10 s), manage dashboard (10 s), voter queue (5 s), results (5 s until closed), ballot (5 s while draft), join page (4 s while waiting). No WebSockets — polling is simple, stateless, and works through any hosting.

api.js — the axios client

One axios instance: `baseUrl: '/api'` (always same-origin — in production the backend serves the frontend; in development the CRA proxy forwards `/api` to port 3000), 15 s timeout, the token interceptor above, and `errorMessage(error, fallback)` which picks the most human-readable message from the server response. `imageUrl(id)` builds image URLs used for candidate photos.

Concept — React concepts used throughout

Components are functions that return JSX (HTML-like syntax). Data flows down through **props**; behavior lives in the component.

Hooks are the way components keep state and talk to the world: `useState` (local state), `useEffect` (run after render — loading data, starting timers), `useContext` (read global state like `useAuth()`), `useRef` (values that survive re-renders without triggering them).

Context (`createContext` + `Provider`) makes a value — the signed-in user — available to every component without passing it down through props.

Polling means re-fetching data on a timer. The alternative, WebSockets, would push updates instantly but needs a persistent connection — more moving parts than a voting app needs.

localStorage stores small strings in the browser permanently (until cleared). The login token lives there so a page refresh keeps the user signed in. Trade-off: any script running on the page can read it (an XSS risk), which is why the app is strict about escaping and headers — and a good topic for section 17.

8.4 Design system

All styling is plain CSS organized in `src/styles/`. `tokens.css` is the design system: a fixed raw palette (`--blue-600`, `--slate-900`, ...), then **semantic tokens** (`--bg`, `--surface`, `--text`, `--primary`, `--success`, `--danger`, ...) that components consume — so theming means changing tokens, never component code. It also defines rank colors for results (`--series-1..4`, chosen to be distinct in hue *and* lightness), elevation shadows, a fluid type scale (clamp-based), Outfit (display) + Work Sans (body) fonts, spacing, radii, and motion durations.

Dark mode is declared twice **on purpose**: inside `@media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) }` (system preference, unless the user explicitly chose light) and in `:root[data-theme="dark"]` (explicit choice wins in both directions). A `prefers-reduced-motion` block zeroes animation durations for users who ask for reduced motion.

8.5 Accessibility

- **Skip link** ("Skip to content") as the first focusable element on every page.
- **Ballot as a real radio group**: a `<fieldset>` with a screen-reader-only legend and `<label>`-wrapped candidate cards — arrow keys work, and screen readers announce "1 of N".
- **Live regions**: the results page is wrapped in `aria-live="polite"` so updated counts are announced.
- Rank numbers printed next to the colored result bars — color is never the only signal.
- Decorative icons are `aria-hidden`; meaningful ones take a `label` prop. Hamburger and theme buttons expose `aria-expanded` / `aria-label`.
- Visible keyboard focus styles, reduced-motion support, and semantic landmarks (`main`, `nav`).

9. Database Design

The database is MongoDB (document-oriented), accessed through Mongoose. Section 7.3 documents each model and index; this section pulls the design decisions together.

Collection	Stores	Key design decision
<code>users</code>	Accounts (email + name)	Identity is just an email — created automatically on first successful sign-in.
<code>elections</code>	Voting sessions	The organizer reference, status machine, join code, and the <code>approvedCount</code> counter that makes the voter cap enforceable atomically.
<code>candidates</code>	Candidates per session	No stored vote totals — counts always come from <code>votes</code> , so concurrent votes never fight over one document.
<code>memberships</code>	Voter ↔ session links	The unique <code>{election, user}</code> index guarantees one join per voter; <code>hasVoted</code> records participation; selfies/face data live here (voter-side), never on the ballot.
<code>votes</code>	Anonymous ballots	No voter reference, random non-time-based id, no timestamps — structurally unable to reveal who chose what.
<code>otpcodes</code>	Active sign-in codes (hashed)	One document per email, HMAC-hashed, 10-minute TTL, attempt counter — auto-deleted by a TTL index.
<code>images</code>	Photos & selfies as Buffers	Binary in MongoDB so images survive deploys on ephemeral-disk hosts; access is enforced by <code>kind</code> at serving time.

Transactions and the replica-set requirement

Voting needs a real transaction (section 7.5), and MongoDB transactions require a **replica set**. In production that is MongoDB Atlas (free tier included). In development, tests, demo mode, and the load test all spin up `mongodb-memory-server`'s single-node replica set (`MongoMemoryReplSet`) so transactions work without any installation. A plain local `mongod` will **not** work — the README calls this out, and `.env.example` documents it.

Why documents instead of tables?

A session and its settings are one document; a voter's link to a session is one document. There are no JOINS: relationships are ObjectId references between collections, resolved with separate lookups (sometimes batched into one aggregation). For this app's shape — mostly independent sessions that never query across each other — documents are a natural fit and keep the data model readable alongside the code.

10. Complete API Reference

All routes live under `/api`. Requests and responses are JSON (candidate photos upload as multipart; image and CSV downloads are binary). Every route except the ones marked "public" requires **Authorization: Bearer <JWT>**.

10.1 System

Method + path	Access	Purpose
GET <code>/api/health</code>	Public	Liveness probe: 200 + database status when ready, 503 while connecting.

10.2 Auth (authRoutes.js)

Method + path	Access	Purpose
POST <code>/api/auth/request-code</code>	Public (rate-limited per IP)	Emails a 6-digit sign-in code. Body: { <code>email</code> , <code>joinCode?</code> }. 429 with <code>retryAfterSeconds</code> if a code was sent < 60 s ago.
POST <code>/api/auth/verify-code</code>	Public (rate-limited per IP)	Exchanges { <code>email</code> , <code>code</code> } for { <code>user</code> , <code>token</code> }. Wrong code → 401 with <code>attemptsLeft</code> ; 5 wrong tries kill the code; each code is single-use.
GET <code>/api/auth/me</code>	Signed in	My profile { <code>id</code> , <code>email</code> , <code>name</code> }.
PUT <code>/api/auth/me</code>	Signed in	Sets my display name (1–80 chars).

10.3 Sessions (electionRoutes.js)

Method + path	Access	Purpose
POST /api/elections	Signed in	Create a session (draft). Body: title, description, maxVoters, selfieMode, resultsVisible, allowList.
GET /api/elections/organizing	Signed in	Sessions I organize, with pending/voted counts.
GET /api/elections/voting	Signed in	Sessions I joined as a voter.
GET /api/elections/join/:code	Public (optional token)	Join-page info: public fields, full , isOrganizer , my membership.
POST /api/elections/join/:code	Signed in	Join a session. Body (face-match only): { faceDescriptor , selfie }. Allow-listed emails auto-approve.
GET /api/elections/:id/ballot	Approved voter	Ballot: session + candidates without vote counts.
POST /api/elections/:id/vote	Approved voter, session open	Cast the vote. Body: { candidateId , selfie? , faceDescriptor? } per selfie mode. 409 if already voted / session not open.
GET /api/elections/:id/results	Organizer always; approved voter when visible	{ election , candidates , totalVotes }. 403 { hidden:true } while results are after-close-only.
GET /api/elections/:id/manage	Organizer	Full manage view: session, allow-list, candidates, counts.
PUT /api/elections/:id/manage	Organizer	Edit settings (selfie mode locks after anyone joins; maxVoters never below approvedCount).
POST /api/elections/:id/manage/open	Organizer	Open voting (needs ≥ 2 candidates).
POST /api/elections/:id/manage/close	Organizer	Close voting (irreversible).
DELETE /api/elections/:id/manage	Organizer	Delete the session and all its data.
POST /api/elections/:id/manage/candidates	Organizer, draft	Add a candidate (multipart image optional, ≤ 1 MB).
PUT /api/elections/:id/manage/candidates/:candidateId	Organizer, until close	Edit candidate name/party/photo.

Method + path	Access	Purpose
DELETE <code>/api/elections/:id/manage/candidates/:candidateId</code>	Organizer, draft	Remove a candidate.
GET <code>/api/elections/:id/manage/voters</code>	Organizer	Voter roster (selfie ids; never face descriptors).
POST <code>/api/elections/:id/manage/voters/approve-all</code>	Organizer	Approve pending oldest-first until cap → { <code>approved</code> , <code>full</code> , <code>remaining</code> }.
POST <code>/api/elections/:id/manage/voters/:membershipId/approve</code>	Organizer	Approve one voter (409 when full).
POST <code>/api/elections/:id/manage/voters/:membershipId/reject</code>	Organizer	Reject one voter (frees a slot; refused after they voted).
POST <code>/api/elections/:id/manage/allow-list</code>	Organizer	Add emails; auto-approves already-waiting listed users.
POST <code>/api/elections/:id/manage/purge-selfies</code>	Organizer, closed	Delete all selfies + face data in the session.
GET <code>/api/elections/:id/manage/results.csv</code>	Organizer	Final results as CSV (formula-injection-safe).

10.4 Images (imageRoutes.js)

Method + path	Access	Purpose
GET <code>/api/images/:id</code>	Candidate photos: public. Selfies: the session's organizer only	Serves the stored bytes with the original content type. Candidate photos cache immutably; selfies are no-store.

11. Security Measures

Security in this app is a series of small, specific decisions. Each row below names one measure, where it lives, and what it defends against.

Measure	Where	Defends against
OTP codes stored only as HMAC-SHA256 hashes	<code>authRoutes.js</code> + <code>otpCode.js</code>	A database leak revealing live sign-in codes.
Constant-time code comparison (<code>timingSafeEqual</code>)	<code>authRoutes.js</code>	Timing attacks that guess the code character by character.
Atomic attempt counting, 5-try lockout, 10-min expiry, single-use codes	<code>authRoutes.js</code>	Brute-force guessing of the 6-digit code.
Rate limits: 600/15 min per IP on sign-in; 300/min per user; 20/min on voting; 20/h on creating sessions	<code>authRoutes.js</code> , <code>electionRoutes.js</code>	Abuse, brute force, accidental spam. Per-user limits because classrooms share an IP.

Measure	Where	Defends against
JWT with 12-hour expiry; server refuses to boot without <code>JWT_SECRET</code>	<code>jwt.js</code> , <code>server.js</code>	Forged tokens; leaked tokens living forever.
One-vote guarantee: conditional atomic update inside a transaction	<code>electionRoutes.js</code> (vote) + tests	Double votes from retries, double-clicks, or parallel requests.
Secret ballot: no voter reference, random non-time-based id, no timestamps on <code>Vote</code>	<code>models/vote.js</code>	The app itself being able to reveal who chose what.
<code>select: false</code> on <code>allowList</code> and <code>faceDescriptor</code> ; descriptors never serialized	<code>models/election.js</code> , <code>models/membership.js</code>	Sensitive fields leaking through ordinary reads or API responses.
Selfies only to the session's organizer, <code>Cache-Control: no-store</code> ; purge button after close	<code>imageRoutes.js</code> , <code>purge-selfies</code>	Selfie photos leaking to anyone else or lingering after the vote.
CSV formula-injection escaping	<code>electionRoutes.js</code> (csvCell)	Crafted names executing as spreadsheet formulas on the organizer's machine.
Email content sanitized: control chars stripped, 80-char cap, link patterns dropped	<code>mailer.js</code>	Phishing links or header injection smuggled through user-typed titles/names.
helmet headers (CSP intentionally off), compression, 400 kb JSON limit	<code>app.js</code>	Common web attacks and oversized payloads.
No CORS — same-origin only	<code>app.js</code> (by design)	Another website calling the API with a victim's token (the token is only ever sent to the same origin).
Upload type + size enforcement (jpeg/png/webp; 1 MB / 200 KB)	<code>electionRoutes.js</code> , <code>m multer</code>	Malicious or oversized files.
<code>trust proxy</code> + client-address normalization	<code>app.js</code> , <code>clientAddress.js</code>	Wrong client IPs breaking rate limits behind a proxy.
Database URI redacted in logs; secrets never in code; graceful shutdown; exit-on-DB-failure	<code>server.js</code> , <code>deploy/azure.js</code>	Secret leakage in logs; half-written state on shutdown; a server running with no database.
Organizer/voter authorization on every endpoint (<code>requireOrganizer</code> , <code>requireApprovedVoter</code>)	<code>access.js</code>	Voters managing sessions, outsiders reading ballots.

11.1 Known limitations (honesty section)

- **Face match has no liveness check** — a printed photo can pass (stated in the README and UI).
- **The ballot is anonymous to the app, not to a database administrator** — someone with direct DB/log access could attempt timing correlation.
- **When SMTP is unconfigured, codes print to the server terminal** — fine for development, never acceptable in production (the server warns at startup).

- **The JWT lives in localStorage** — standard for small apps, but vulnerable to XSS (any script on the page can steal it). Section 17 lists the stronger alternative.

12. Testing

The backend has a real automated test suite — 28 tests in `backend/test/api.test.js` — run with the Node built-in test runner (`node --test`) plus **supertest** (makes HTTP requests against the Express app in-process) and **mongodb-memory-server** (a throwaway in-memory MongoDB **replica set**, so transactions work). The mailer is stubbed: tests capture the codes instead of sending email. Nothing ever touches real data.

```
cd backend
npm test          # run all API tests
npm run load-test # 400 simulated voters over 4 sessions (perf + correctness)
```

12.1 What the tests verify

Area	Tests	Highlights
Sign-in	7	Correct code signs in and is single-use; resend blocked for 60 s (429); lockout after 5 wrong tries; expired code rejected; join-link emails name the session and strip links; IP normalization through a proxy; missing token refused.
Sessions (create/manage)	4	Only the organizer can manage; opening needs ≥ 2 candidates and candidates lock afterwards; invalid settings refused; at most 10 active sessions per organizer.
Joining & approval	5	Allow-listed emails auto-approve; adding an email later approves a waiting voter; 20 simultaneous approvals against a cap of 5 approve exactly 5; approve-all stops at the cap; rejection frees a place.
Voting	6	Only approved voters, only while open, only once; the stored ballot contains only <code>_id</code> , <code>candidate</code> , <code>election</code> (anonymity check); 50 simultaneous requests produce exactly 1 vote ; 100 concurrent voters are all counted; 4 concurrent sessions stay separate; a voter in one session cannot touch another.
Results	2	Results hidden until close when configured; deleting a session cascades to all its data.
Selfies	3	Photo mode requires a selfie and only the organizer can view it; face match rejects a different face and accepts the enrolled one; candidate photos are public.

12.2 Load test, demo mode, and helpers

- **Load test** (`backend/scripts/load-test.js`): simulates N voters (default 400) across 4 sessions, each arriving randomly within a 3-second window and going through join → ballot → vote → results. It reports median/95th/99th/max response times per endpoint, throughput, and failure counts — and then **verifies every session's vote count and hasVoted count exactly match**, exiting non-zero on any mismatch. This is both a performance probe and a correctness stress test.

- **Demo mode** (`npm run demo`): starts the whole app on a temporary in-memory replica set — no Atlas needed. All data disappears when it stops; meant for trying the app, never for a real vote.
- **test-email** (`npm run test-email -- you@example.com`): verifies the SMTP configuration by sending a real test code, with Gmail app-password troubleshooting hints.
- **reset-db** (`npm run reset-db [-- --yes]`): drops only the legacy v1 `users/candidates` collections, and only after confirming the v1 `aadharCardNumber_1` index exists — dry-run by default (section 15).

13. Running the Project Locally

13.1 Prerequisites

- **Node.js 18 or newer** (built and tested on newer versions).
- A **MongoDB Atlas** database — the free tier is enough. (Required because of transactions; see section 9. A plain local MongoDB will not work.)

13.2 Setup steps

Install everything: `npm run install-all` (root, backend, and frontend dependencies in one command).

Copy `backend/.env.example` to `backend/.env` and fill in `MONGODB_URI` (Atlas connection string) and `JWT_SECRET` (any long random string).

Create `frontend/.env` containing `PORT=3001`. The backend runs on 3000 and the React dev server proxies `/api` to it.

Start both servers: `npm run dev` (runs backend + frontend together via `concurrently`).

Open `http://localhost:3001`.

Trying it on a phone on the same Wi-Fi: open `http://<your computer's IP>:3001`. Note that selfie sessions need the camera, which phone browsers only allow over HTTPS or on localhost — test those on the computer or on the deployed site.

Without a database at all: `npm run demo` — everything runs on a temporary in-memory database and is lost on stop.

13.3 Sending sign-in codes by email

Until email is configured, sign-in codes print to the backend terminal — good enough to start. For real email, create a dedicated Gmail mailbox for the app, enable 2-Step Verification on it, generate an **app password** (`myaccount.google.com/apppasswords`), and set in `backend/.env`:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=the-app-mailbox@gmail.com
SMTP_PASS=its-16-letter-app-password
```

Then restart the backend and check with `cd backend && npm run test-email -- you@gmail.com`. (`MAIL_FROM` optionally overrides the sender address.)

13.4 Environment variables

Variable	Needed	Purpose
<code>MONGODB_URI</code>	Yes	MongoDB connection string. Must be a replica set (Atlas is).
<code>JWT_SECRET</code>	Yes	Long random string. Signs login tokens and keys the OTP hash — the server refuses to start without it.
<code>PORT</code>	No	Listen port. Default 3000.
<code>NODE_ENV</code>	No	<code>production</code> when deployed (changes trust-proxy behavior).
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>MAIL_FROM</code>	For real email	SMTP settings for the sign-in codes. Without them, codes print to the server console.
<code>FACE_MATCH_THRESHOLD</code>	No	Face-match strictness. Default 0.5; lower is stricter.
<code>MONGODB_POOL_SIZE</code>	No	MongoDB connection pool size. Default 50.

14. Deployment

Because the backend serves the built frontend (section 5), deployment is **one Node service**. Two targets are supported: Azure App Service (automated with a script) and Render (blueprint file).

14.1 Azure App Service

`deploy/azure.js` automates the whole Azure workflow. Requires the Azure CLI and a working `backend/.env`.

`az login`, then in MongoDB Atlas → Network Access allow `0.0.0.0/0` (Azure connects from changing addresses; the database still requires its credentials).

One-time: `npm run azure:setup -- <app-name>` — creates resource group `voting-app-rg` and a free **F1 Linux plan** (default region `indiasouthcentral`, one of the few Azure for Students allows), then the web app on Node 24 LTS. It copies database/email settings from `backend/.env` and generates a new random `JWT_SECRET` (48 random bytes), passing secrets through a temp file rather than the command line. If the region is refused by policy, it lists the allowed ones — rerun with `--location`.

Every update: `npm run azure:deploy` — builds the frontend, packages the backend (excluding `.env`, tests, `node_modules`, uploads), uploads the zip, then polls `GET /api/health` until the database reports connected.

Without the Azure CLI (VS Code extension or portal), `npm run azure:package` writes `deploy/package/` (the folder to deploy), `deploy/app.zip`, and `deploy/app-settings.env` (all settings, git-ignored — it contains secrets) without uploading anything.

Free-tier reality: the app sleeps after ~20 minutes without visitors (next visitor waits up to a minute) and gets 60 minutes of CPU per day. For a real voting day, scale to Basic B1 (always-on) and back afterwards — the README has the exact commands. The current live target is <https://nnm-voting-app.azurewebsites.net> (recorded in the git-ignored `.azure-app.json`).

14.2 Render

`render.yaml` describes the same service as a Render Blueprint: build `cd backend && npm install && cd ../frontend && npm install && npm run build`, start `node backend/server.js`, health check `/api/health`. Set `MONGODB_URI`, `JWT_SECRET`, `NODE_ENV=production` and the `SMTP_*` values in the dashboard. Use an always-on instance — a sleeping free instance makes the first voters wait while it wakes up.

14.3 Notes

- `vercel.json` at the root is a leftover from the v1 deployment attempts and is not used.
- Deploy artifacts (`deploy/app.zip`, `deploy/package/`, `deploy/app-settings.env`, `.azure-app.json`) are git-ignored — secrets never enter the repository.

15. Project History and Legacy Code

15.1 How the project evolved

From the git history: the project began as **v1** — an "online voting system" with Aadhaar-number registration and password login, admin/voter **roles**, and candidate CRUD where votes were counters stored on each candidate document. It went through React frontend iterations, Vercel deployment attempts, and path fixes, and was then **rebuilt as v2** — the current app: passwordless OTP sign-in, voting **sessions** with organizers and voter limits, transactional anonymous ballots, and selfie/face-match options. The v1 files were never deleted from the repository; the v2 server simply does not load them.

15.2 Files that are NOT part of the live app

None of these are imported by `backend/app.js` / `server.js` or by any live component. They exist for history (and a few still hold surprises):

Location	Files	What they were
<code>backend/routes/</code>	<code>userRoutes.js</code> , <code>candidates.js</code> , <code>candidateRoute.js</code>	v1 Aadhaar+password signup/login and admin candidate CRUD with disk uploads. (<code>userRoutes.js</code> even contains a leftover merge-conflict line from an old <code>git merge --abort</code> .)
<code>backend/middleware/</code>	<code>auth.js</code> , <code>admin.js</code> , <code>authMiddleware.js</code>	v1 role-based middleware (<code>admin/voter</code>). One contains a hardcoded fallback secret — dead code, but a good lesson in why such things should never exist.
<code>backend/models/</code>	<code>modelIndex.js</code>	v1 model registry (role, Aadhaar, password fields).
<code>backend/</code> root	<code>db.js</code> , <code>config/database.js</code> , <code>config/gridfs-config.js</code> , <code>atlas.js</code> , <code>atlas-test.js</code> , <code>connectionTest.js</code> , <code>db-test.js</code> , <code>test-connection.js</code> , <code>test-mongo-connection.js</code> , <code>testRoutes.js</code> , <code>mongo-whitelist-helper.js</code> ,	v1 connection debugging scripts, GridFS setup, disk uploads, and the old package manifest (with <code>bcrypt</code> , <code>cors</code> , <code>jest</code> — none used in v2).

Location	Files	What they were
	<code>install-dependencies.js</code> , <code>package.json.bak</code> , <code>image-upload-config.md</code> , <code>uploads/</code>	
<code>frontend/src/</code>	<code>api.js</code> , <code>utils/apiDebug.js</code> , <code>utils/debugHelper.js</code> , <code>utils/NetworkDiagnostic.js</code>	Pre-refactor API client and network-debug helpers (the live client is <code>utils/api.js</code>).
<code>frontend/</code>	<code>middleware/</code> , <code>models/</code> directories	Mistakenly placed v1-style folders, unused.
Root	<code>client/</code> , <code>server/</code> , <code>routes/</code> , <code>src/</code> , <code>config/</code> , <code>public/</code> , <code>vercel.json</code> , <code>api-tests.http</code> , <code>cleanup.js</code> , <code>scripts/postbuild.js</code> (empty), a stray <code>.js</code> file and a screenshot	v1 application folders and deployment experiments that predate the <code>backend/</code> + <code>frontend/</code> split.
Git history	Deleted: <code>AdminDashboard.js</code> , <code>CandidateCard.js</code> , <code>HomePage.js</code> , <code>Register.js</code> , <code>ServerStatus.js</code> , <code>index.css</code> ...	v1 React components, superseded by the current components (section 8.2).

Why they matter even though they are dead: the v1 `users` collection left in an old database contains a unique `aadharCardNumber_1` index that blocks new sign-ins — the server prints a startup warning, and `npm run reset-db` removes exactly those collections (dry-run first, only when the legacy index is detected). The files themselves are safe to archive or delete.

16. Concept Glossary

Every important term from this project in one place, each in a sentence or two.

Term	Meaning in this project
API / REST	The HTTP interface the frontend uses: URLs identify resources, HTTP methods name actions, JSON carries data.
JSON	The JavaScript data format used for every request and response body.
JWT	A signed login token (<code>header.payload.signature</code>) valid 12 h; proves identity without a session store.
Bearer token	The way the token is sent: the <code>Authorization: Bearer <token></code> header.
OTP	A 6-digit code that works once and expires in 10 minutes.
HMAC	<code>HMAC(key, message)</code> — an unforgeable fingerprint; used to store OTPs as hashes.
Timing-safe comparison	Comparing secrets in constant time so response speed leaks nothing.
Rate limiting	Capping requests per client per time window (per IP or per user).
Middleware	Functions Express runs on each request, in order: parsing, auth, limits, handlers.
Route / handler	A URL pattern plus the function that serves it.
Schema / model	The shape of a MongoDB document, and the object used to read/write it.

Term	Meaning in this project
Collection / document	MongoDB's equivalent of a table and a row — but JSON-like.
Index	A database-side lookup table for a field; makes queries fast and enforces uniqueness.
TTL index	An index that makes MongoDB auto-delete a document when a date passes (used for OTP expiry).
Replica set	A cluster of MongoDB nodes that keep copies of data — required for transactions.
Transaction / atomicity	Grouping writes so they all happen or none do; no half-done state is ever visible.
Aggregation pipeline	Staged data processing inside MongoDB (match → group → sort) — used to count votes.
Race condition	A bug from two operations interleaving; fixed here with single conditional atomic updates.
SPA	Single-page app: one HTML page, JavaScript renders every screen; routes exist client-side.
SPA fallback	The server answering every non-API path with <code>index.html</code> so deep links survive reloads.
Proxy	An intermediary forwarding requests; the dev server proxies <code>/api</code> to port 3000.
Polling	Re-fetching data on a timer — the app's mechanism for live updates.
localStorage	Browser storage that survives refreshes; holds the login token.
Component / prop / hook	React building blocks: UI functions, the data passed into them, and <code>useState/useEffect/useContext</code> for state, effects, and global values.
Context	A React mechanism making one value (the signed-in user) available everywhere.
CSS custom properties	<code>--var</code> tokens defining the design system's colors, spacing, and type; the dark theme just re-declares them.
Media query / prefers-reduced-motion	CSS that adapts to the device (and to the user's reduced-motion preference).
Data URL	A file encoded as text (<code>data:image/jpeg;base64,...</code>) — how selfies travel in JSON.
SMTP	The protocol for sending email; the app sends codes through Gmail's SMTP server.
App password	A generated 16-letter password granting one app access to a Google account.
Environment variable	A per-machine setting (<code>MONGODB_URI</code> , <code>JWT_SECRET</code> ...) loaded from <code>.env</code> , keeping secrets out of code.
npm script	A named command in <code>package.json</code> (<code>npm run dev</code> , <code>npm test</code>).
Euclidean distance	The straight-line distance between two face descriptors; small = same face.

Term	Meaning in this project
CSP / helmet	Security headers an app sends; helmet sets them (CSP is off here for the face model's inline scripts).

17. Ideas for Further Learning

The project is deliberately complete yet simple. Each of these is a natural next step that teaches a real-world technique.

- **Real-time updates with WebSockets** (Socket.IO): replace polling with server-pushed events — instant results, "voting is now open" moments.
- **Stronger auth**: refresh tokens with short-lived access tokens, httpOnly Secure cookies instead of localStorage, or proper OAuth.
- **Password logins done right**: bcrypt/argon2 hashing, salt rounds, password-reset flows.
- **Liveness detection** for face match: blink/turn challenges so a photo cannot pass.
- **TypeScript**: types across both backend and frontend — catches bugs at compile time.
- **React Query** (TanStack Query): replaces hand-rolled polling with caching, retries, and background refetching.
- **Component tests** with React Testing Library (already installed by CRA) and end-to-end tests with Playwright.
- **CI/CD**: GitHub Actions running the tests on every push and deploying automatically.
- **Containerization**: a Dockerfile producing one image for any host.
- **Redis** for rate limiting and caching across restarts, and structured logging/monitoring.
- **Pagination & search** in the session lists, internationalization (i18n), and audit logs for organizer actions.

Appendix A — npm Scripts

Root package.json

Command	What it does
<code>npm run dev</code>	Runs backend (nodemon, port 3000) and frontend (CRA, port 3001) together.
<code>npm run demo</code>	Same, but with a temporary in-memory MongoDB — no setup, data lost on stop.
<code>npm run install-all</code>	Installs dependencies in root, backend, and frontend.
<code>npm run build</code>	Builds the frontend production bundle into <code>frontend/build</code> .
<code>npm test</code> / <code>npm run load-test</code>	Backend API tests / simulated 400-voter load test.
<code>npm run azure:setup</code> / <code>azure:deploy</code> / <code>azure:package</code>	Azure deployment automation (section 14).

Backend package.json

Command	What it does
<code>npm start / npm run dev</code>	Run the server / run with nodemon auto-restart.
<code>npm test</code>	Run the API test suite (<code>node --test test/api.test.js</code>).
<code>npm run demo</code>	Start on a temporary in-memory replica set.
<code>npm run load-test</code>	Simulate 400 voters across 4 sessions (args: voters, sessions).
<code>npm run reset-db / -- --yes</code>	Drop legacy v1 collections — dry-run unless confirmed.
<code>npm run test-email -- <address></code>	Verify SMTP by sending a test sign-in code.

Frontend package.json

Command	What it does
<code>npm start</code>	Run the CRA dev server (prestart copies face models into <code>public/models</code>).
<code>npm run build</code>	Build the production bundle (prebuild copies face models too).
<code>npm test</code>	CRA's React test runner.

Appendix B — Dependencies

Backend

Package	Version	Used for
express	^4.18.2	The web framework.
mongoose	^7.0.0	MongoDB object modeling (schemas, models, indexes).
jsonwebtoken	^9.0.0	Signing/verifying login tokens.
nodemailer	^6.9.14	Sending the sign-in code email via SMTP.
multer	^2.0.2	Multipart uploads (candidate photos, selfies).
express-rate-limit	^7.4.0	Rate limiting per IP and per user.
helmet	^7.1.0	Security headers.
compression	^1.7.4	Gzip responses.
dotenv	^16.0.3	Loads <code>.env</code> into environment variables.
(dev) mongodb-memory-server	^9.4.1	In-memory replica set for tests/demo/load test.
(dev) supertest	^7.0.0	HTTP testing of the Express app.
(dev) nodemon	^2.0.21	Auto-restarts the server during development.

Frontend

Package	Version	Used for
react / react-dom	^18.2.0	The UI library.
react-router-dom	^6.8.1	Client-side routing.
react-scripts	5.0.1	Create React App build/dev/test tooling.
axios	^1.3.4	HTTP client with interceptors.
react-toastify	^9.1.3	Toast notifications.
qrcode.react	^4.2.0	Join QR code.
@vladmandic/face-api	^1.7.15	In-browser face detection/descriptors (lazy-loaded from /models).
web-vitals	^2.1.4	Performance metrics.
(dev) @testing-library/*	^13–14	React component-testing utilities (CRA default).

Closing Note

This document was generated from the project's actual source code and configuration — every endpoint, field, limit, and behavior described here exists in the repository. The best way to learn the project is to read this document next to the code: open [backend/routes/electionRoutes.js](#) while reading section 7.5, or [frontend/src/components/VotingPage.js](#) while reading section 8.2.

Enjoy the project — it was built to be understood.